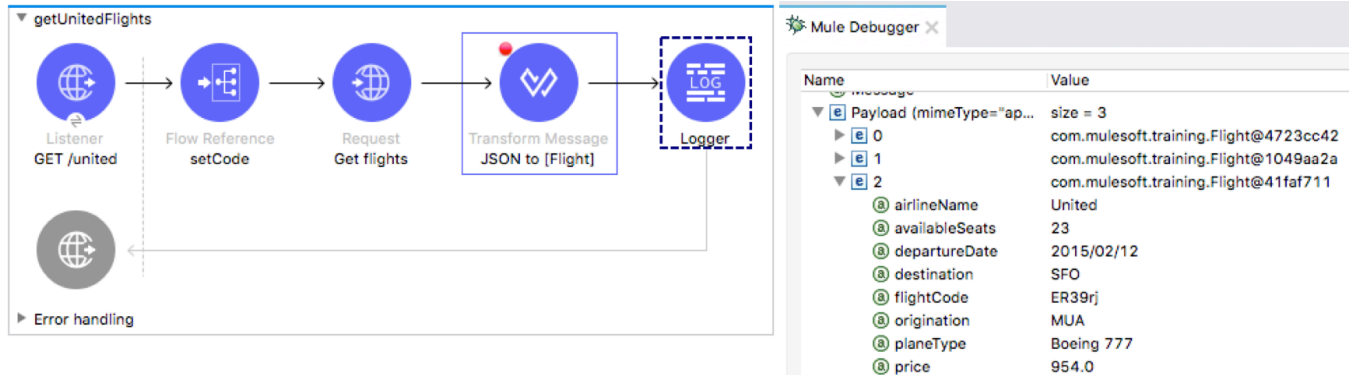


Walkthrough 8-4: Transform data from multiple services to a canonical format

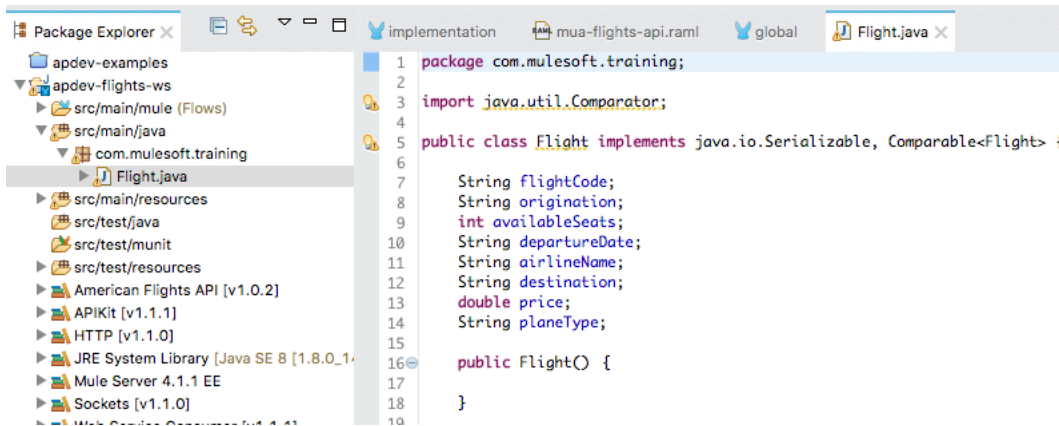
In this walkthrough, you will transform the JSON returned from the American and United web services and the SOAP returned from the Delta web service to the same format. You will:

- Define a metadata type for the Flight Java class.
- Transform the results from RESTful and SOAP web service calls to a collection of Flight objects.



Review Flight.java

1. Return to apdev-flights-ws in Anypoint Studio.
2. Open Flight.java in src/main/java and review the file.



3. Close the file.

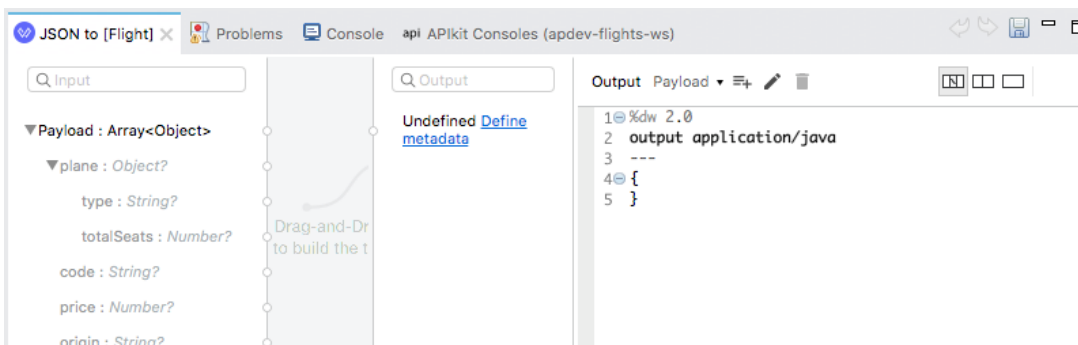
Change the American flow to return Java Flight objects instead of JSON

4. Return to getAmericanFlights in implementation.xml.
5. Add a Transform Message component to the end of the flow.
6. Add a Logger at the end of the flow.

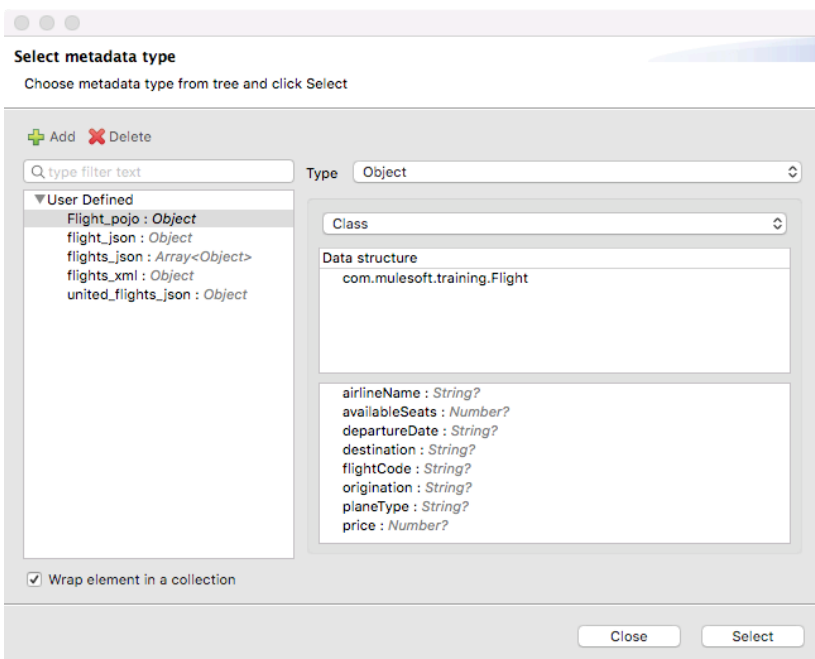
7. Change the name of the Transform Message component to JSON to [Flight].



8. In the Transform Message properties view, look at the input section in the Transform Message properties view; you should see metadata already defined.



9. In the output section of the Transform Message properties view, click the Define metadata link.
10. In the Select metadata type dialog box, select the user-defined Flight_pojo.
11. Select Wrap element in a collection in the lower-left corner.



-
- The screenshot displays the IntelliJ IDEA IDE interface for the 'JSON to [Flight]' API. The left sidebar shows the 'Payload' structure as an 'Array<Object>' with a 'plane' object containing fields like 'type', 'totalSeats', 'code', 'price', 'origin', 'destination', 'ID', 'departureDate', and 'emptySeats'. The right sidebar shows the 'Array<Object>' structure with fields like 'airlineName', 'availableSeats', 'departureDate', 'destination', 'flightCode', 'origination', 'planeType', and 'price'. A central diagram illustrates the mapping between the JSON fields and the Flight object fields. The bottom pane shows the generated Java code for the Flight object, including the payload map and the class name 'com.mulesoft.training.Flight'.

- ```
1 %dw 2.0
2 output application/java
3 ---
4 payload map (payload01 , indexOfPayload01) ->
5 airlineName: "American",
6 availableSeats: payload01.emptySeats.
```

16. Save to redeploy the project.
17. In Advanced REST Client, change the URL and make a request to `http://localhost:8081/american`; you should see a representation of a collection of Java objects.



## Debug the application

18. Return to Anypoint Studio.
19. Stop the project.
20. Add a breakpoint to the Get flights operation.
21. Debug the project.
22. In Advanced REST Client, make another request to `http://localhost:8081/american`.
23. In the Mule Debugger, step to the Transform message component and examine the payload.

The screenshot shows the Mule Debugger interface. On the left, the 'payload' is expanded, showing a JSON array with one object. The object has fields: 'ID' (5), 'code' ('rree1093'), 'price' (142), and 'departureDate' ('2016-02-11T00:00:00'). The 'vars' section shows a map with 'code' set to 'SFO' and 'mediaType' set to 'application/java; charset=UTF-8'. On the right, the full JSON payload is displayed:

```
[{"ID": 5, "code": "rree1093", "price": 142, "departureDate": "2016-02-11T00:00:00", "origin": "MUA", "destination": "SFO", "emptySeats": 1, "plane": {"type": "Boeing 737", "totalSeats": 150}}]
```

24. Step to the Logger and look at the payload it should be a collection of Flight objects.

The screenshot shows the Mule Debugger interface. On the left, the 'payload' is expanded, showing a collection of Flight objects. The objects are: com.mulesoft.training.Flight@2615c58b, com.mulesoft.training.Flight@3ffa16ad, com.mulesoft.training.Flight@410afd9, com.mulesoft.training.Flight@2c75f8fa, and com.mulesoft.training.Flight@24c45962. The 'vars' section shows a map with 'code' set to 'SFO' and 'mediaType' set to 'application/java; charset=UTF-8'. On the right, the full JSON payload is displayed:

```
[{"ID": 5, "code": "rree1093", "price": 142, "departureDate": "2016-02-11T00:00:00", "origin": "MUA", "destination": "SFO", "emptySeats": 1, "plane": {"type": "Boeing 737", "totalSeats": 150}}]
```

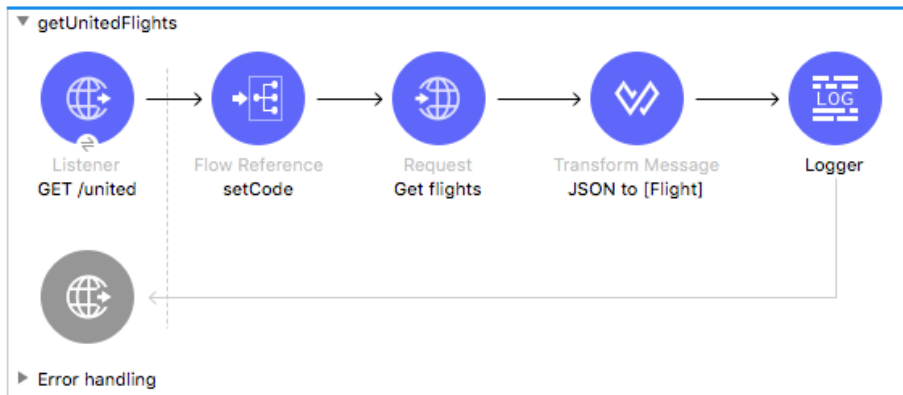
Below the debugger, the application flow diagram is shown. The flow starts with a 'Listener GET /american', followed by a 'Flow Reference setCode', then a 'Get flights' component (which has a red dot indicating a breakpoint), then a 'Transform Message JSON to [Flight]', and finally a 'Logger' component (which is highlighted with a dashed box).

25. Step through the rest of the application and switch perspectives.

## Change the United airline flows to return Java Flight objects instead of JSON

26. Return to geUnitedFlights in implementation.xml.

27. Add a Transform Message component at the end of the flow.
28. Change the name of the Transform Message component to JSON to [Flight].
29. Add a Logger to the end of the flow.



30. In the Transform Message properties view, look at the input section; you should see metadata already defined.
31. In the output section of the Transform Message properties view, click the Define metadata link.
32. In the Select metadata type dialog box, select the user-defined Flight\_pojo.
33. Select Wrap element in a collection in the lower-left corner.
34. Click Select; you should now see output metadata in the output section of the Transform Message properties view.

JSON to [Flight] x Problems Console api APIkit Consoles (apdev-flights-ws)

Input

Payload : Object

flights : Array<Object>?

- airlineName : String?
- price : Number?
- departureDate : String?
- planeType : String?
- origin : String?
- code : String?
- emptySeats : Number?

Output

Array<Object>

- airlineName : String?
- availableSeats : Number?
- departureDate : String?
- destination : String?
- flightCode : String?
- origination : String?
- planeType : String?
- price : Number?

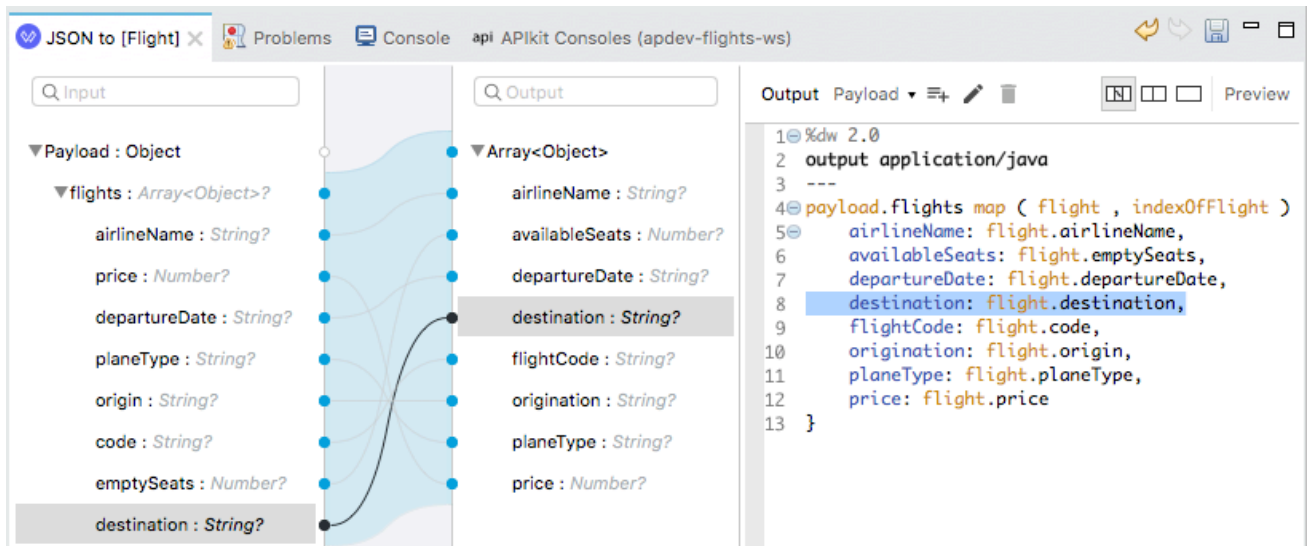
Output Payload

```

1 %dw 2.0
2 output application/java
3 ---
4 {
5 }

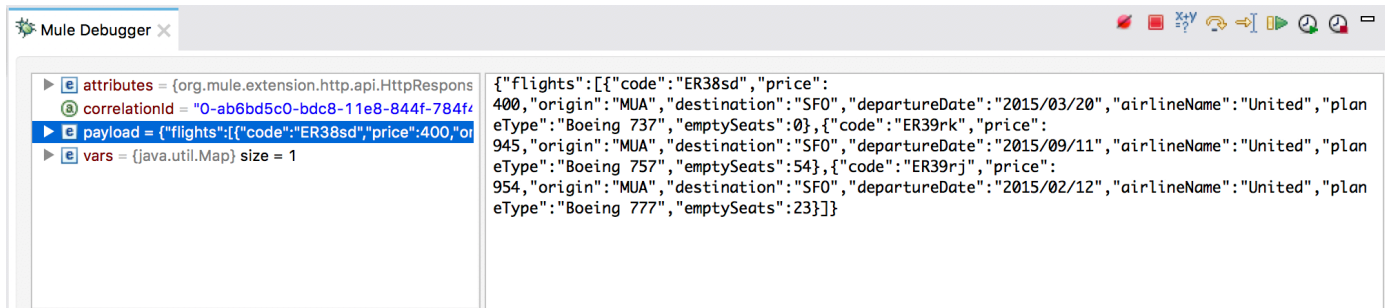
```

35. Map fields by dragging them from the input section and dropping them on the corresponding field in the output section.



## Debug the application

36. Add a breakpoint to the Transform Message component.
37. Save the files to redeploy the project.
38. In Advanced REST Client, change the URL to make a request to <http://localhost:8081/united>.
39. In the Mule Debugger, examine the payload.



40. Step to the Logger and look at the payload it should be a collection of Flight objects.

The screenshot shows the Mule Debugger interface. The top pane displays the payload details for a message. The payload is a collection of Flight objects. The bottom pane shows a flow diagram for the `getUnitedFlights` process, which includes a Listener, Flow Reference, Request, Transform Message, and Logger component.

**Mule Debugger**

**Attributes:**

- `attributes` = {org.mule.extension.http.api.HttpResponse}
- `correlationId` = "0-ab6bd5c0-bdc8-11e8-844f-784f4"
- `payload` = {java.util.ArrayList} size = 3
  - `0` = {com.mulesoft.training.Flight} com.mulesoft.tr
  - `1` = {com.mulesoft.training.Flight} com.mulesoft.tr
  - `2` = {com.mulesoft.training.Flight} com.mulesoft.tr
- `^mediaType` = application/java; charset=UTF-8
- `vars` = {java.util.Map} size = 1
  - `code` = "SFO"

**com.mulesoft.training.Flight@77cd2a88**

**implementation** **config.yaml** **global** **application-types.xml**

**getUnitedFlights**

Listener GET /united → Flow Reference setCode → Request Get flights → Transform Message JSON to. [Flight] → Logger

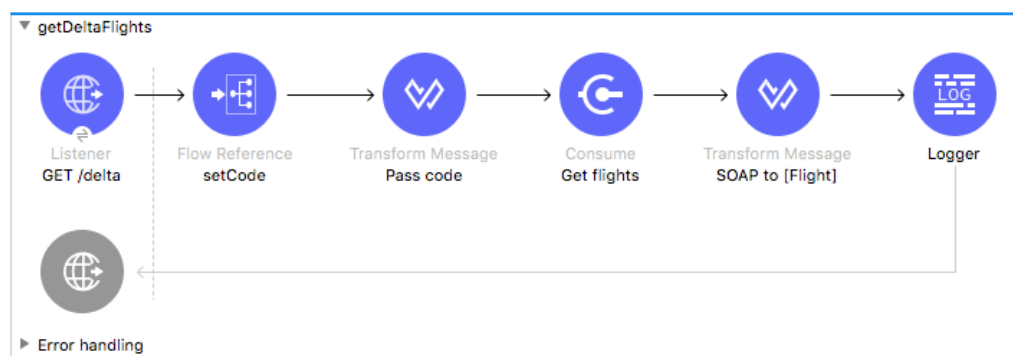
41. Step through the rest of the application and switch perspectives.

## Change the Delta flow to return Java Flight objects

42. Return to `geDeltaFlights` in `implementation.xml`.

43. Change the name of the Transform Message component to SOAP to [Flight].

44. Add a Logger to the end of the flow.



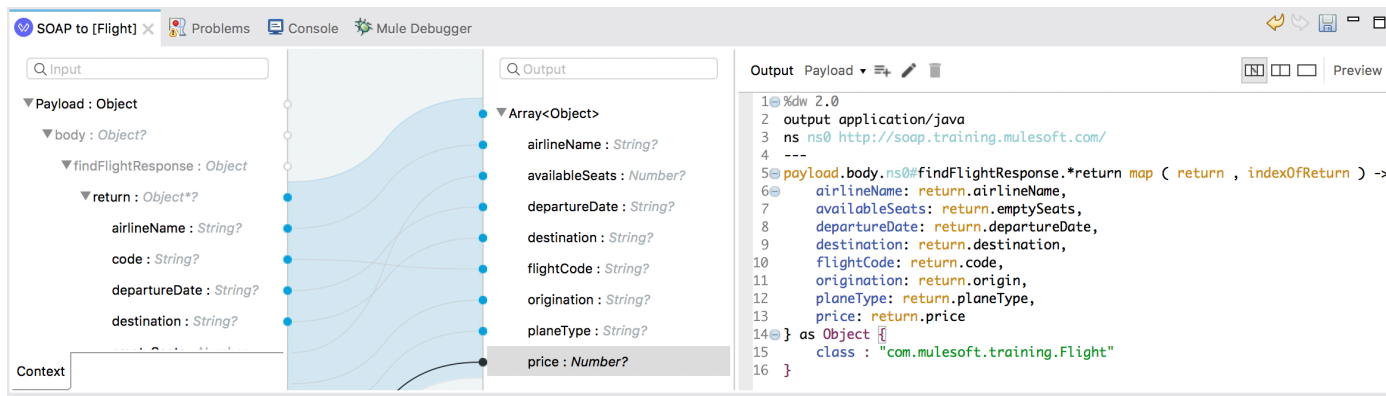
45. Look at the input section in the Transform Message properties view; you should see metadata already defined.

46. In the output section of the Transform Message properties view, click the Define metadata link.

47. In the Select metadata type dialog box, select the user-defined `Flight_pojo`.

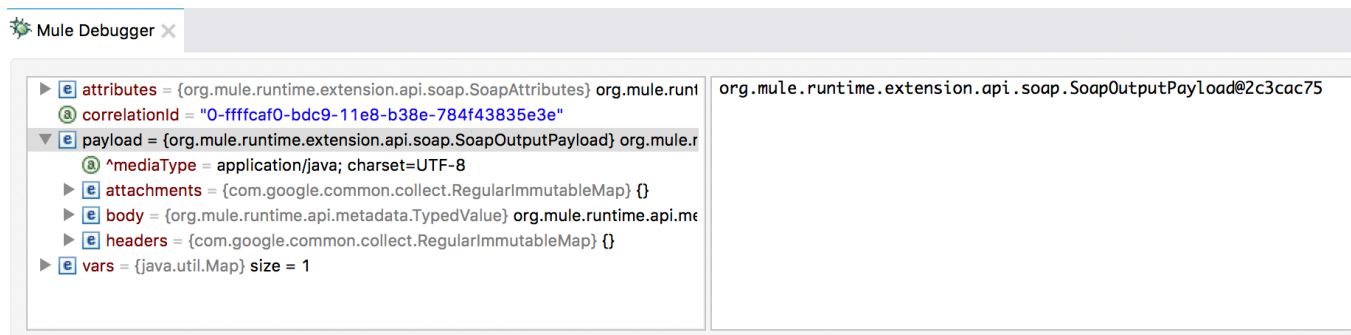


48. Select Wrap element in a collection in the lower-left corner.
49. Click Select; you should now see output metadata in the output section of the Transform Message properties view.
50. Map the fields by dragging them from the input section and dropping them on the corresponding field in the output section.



## Debug the application

51. Add a breakpoint to the Transform Message component.
52. Save the file to redeploy the project.
53. In Advanced REST Client, change the URL to make a request to <http://localhost:8081/delta>.
54. In the Mule Debugger, examine the payload.





55. Step to the Logger and look at the payload it should be a collection of Flight objects.

The screenshot shows the Mule Debugger interface. The top pane displays the message payload, which is a collection of Flight objects. The bottom pane shows the flow diagram for the `getDeltaFlights` process, with the Logger component highlighted.

**Mule Debugger**

Attributes: `{org.mule.runtime.extension.api.soap.S SoapAttributes} org.mule.runt`  
CorrelationId: `"0-ffffcaf0-bdc9-11e8-b38e-784f43835e3e"`  
Payload: `{java.util.ArrayList} size = 3`  
0: `{com.mulesoft.training.Flight} com.mulesoft.training.Flight@32fd1646`  
1: `{com.mulesoft.training.Flight} com.mulesoft.training.Flight@5f78c3c`  
2: `{com.mulesoft.training.Flight} com.mulesoft.training.Flight@2e9e1997`  
MediaType: `application/java; charset=UTF-8`  
Vars: `{java.util.Map} size = 1`

**Implementation** | config.yaml | global | application-types.xml

**getDeltaFlights**

Listener GET /delta → Flow Reference setCode → Transform Message Pass code → Consume Get flights → Transform Message SOAP to [Flight] → **Logger**

56. Step through the rest of the application and switch perspectives.